



HITEC UNIVERSITY, TAXILA

Department of Computer Science

OOP – Project Report

Smart Campus Management System

Course	Object Oriented Programing
Semester	2nd Semester — Spring 2026
Lab Instructor	Moshin Shahzad

Student Name	Roll Number
Abdullah Butt	25-CS-078
Hamid Zia Afridi	25-CS-123
Soban Ali Malik	25-CS-138

1. Introduction

1.1 Problem Statement

Modern universities manage vast amounts of operational data across multiple domains: student records, faculty assignments, course enrollment, library catalogs, fee collection, and hostel allocation. Managing these through separate, disconnected systems introduces data inconsistency, redundancy, and administrative overhead. A unified, object-oriented software system can model the real-world relationships among these entities and provide a robust, maintainable, and extensible platform.

1.2 System Goals

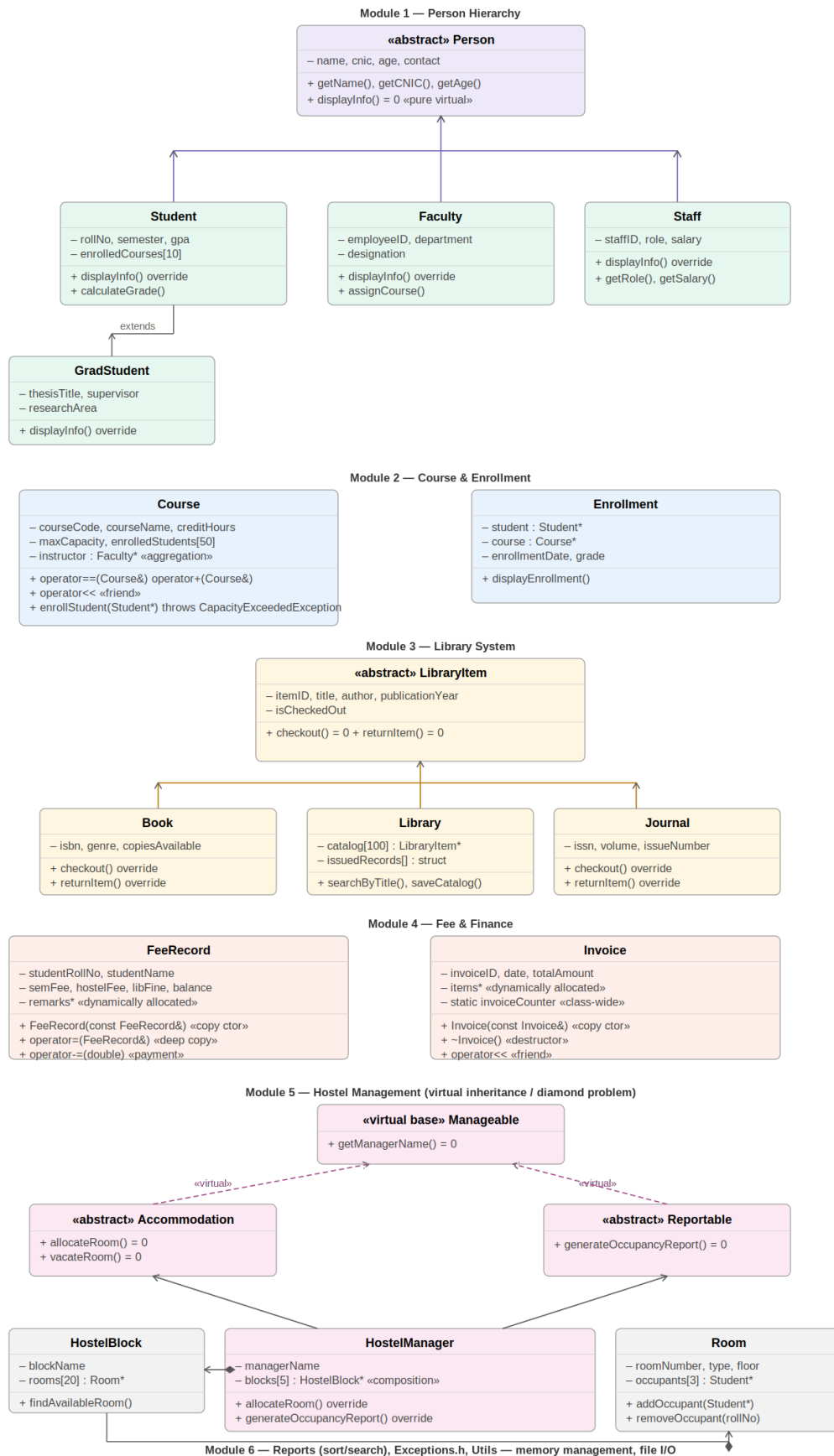
- Design a fully object-oriented C++17 console application covering 6 campus modules.
- Demonstrate all 25 OOP concepts from the CS-104L course checklist.
- Implement clean, well-commented, professionally structured code with no memory leaks.
- Persist library data to disk using `fstream` and restore it on startup.
- Host the project on GitHub with meaningful commit history and a CI/CD workflow.
- Produce a UML class diagram and this project report as required deliverables.

2. System Design

2.1 Architecture Overview

The SCMS is divided into six interconnected modules. Each module is physically separated into its own subdirectory under `src/`, with classes split into `.h` (declaration) and `.cpp` (definition) files following the single-responsibility principle. A central `main.cpp` wires the modules together through a console menu, seeding sample data and demonstrating every OOP concept.

2.2 UML Class Diagram



3. OOP Concepts Implementation

The following section documents all 25 mandatory OOP concepts, explaining where and how each is implemented in the SCMS codebase.

1. Classes & Objects

Every module defines at least one class. **Person**, **Student**, **Course**, **Library**, **FeeRecord**, **Room**, and their relatives are all first-class C++ classes with private data and public interfaces.

```
class Student : public Person {
private:
    string rollNo;
double gpa; public:
    void displayInfo() const override;
};
```

2. Encapsulation (getters/setters)

All data members are private or protected. Access is provided exclusively via public getters and setters, keeping implementation details hidden from the caller.

```
string getRollNo() const { return rollNo; } void
setRollNo(const string& r) { rollNo = r; }
```

3. Constructors

Person, **Course**, and **FeeRecord** each define a parameterised constructor. **FeeRecord** additionally defines a copy constructor (deep copy) and **Invoice** a copy constructor for its dynamic items array.

```
FeeRecord::FeeRecord(const FeeRecord& other)
: studentRollNo(other.studentRollNo), ... {
    remarks = new string[remarkCapacity];
    for (int i=0; i < remarkCapacity; i++) remarks[i] = other.remarks[i];
}
```

4. Destructors

Library iterates its `LibraryItem*` array and calls `delete` on each pointer. **Invoice** deletes its dynamic items array. **HostelBlock** and **HostelManager** delete all dynamically created **Room** and **HostelBlock** objects.

```
Library::~~Library() {
    for (int i=0; i < catalogSize; i++) delete catalog[i];
    catalog[i] = nullptr;
}
```

5. Single Inheritance

Student, **Faculty**, and **Staff** each inherit directly from **Person**, re-using name, CNIC, age, and contact while adding domain-specific attributes.

```
class Student : public Person { ... }; class
Faculty : public Person { ... }; class Staff
: public Person { ... };
```

6. Multi-level Inheritance

GradStudent extends **Student**, which extends **Person**, forming a three-level chain:

GradStudent → Student → Person.

```
class GradStudent : public Student {
string thesisTitle, supervisor;    void
displayInfo() const override; };
```

7. Multiple Inheritance

HostelManager inherits from both **Accommodation** and **Reportable**, combining the room-allocation interface with the reporting interface in one concrete class.

```
class HostelManager : public Accommodation,
public Reportable {    void
allocateRoom(...) override;
    void generateOccupancyReport() const override;
};
```

8. Virtual Inheritance (Diamond Problem)

Both **Accommodation** and **Reportable** inherit from **Manageable** using virtual inheritance. This ensures that **HostelManager** has exactly one copy of **Manageable**'s subobject, resolving the diamond ambiguity.

```
class Manageable { virtual string getManagerName()=0; };
class Accommodation : public virtual Manageable { ... };
class Reportable : public virtual Manageable { ... };
class HostelManager : public Accommodation, public Reportable { ... };
```

9. Abstract Classes & Pure Virtual Functions

Person, **LibraryItem**, **Accommodation**, **Reportable**, and **Manageable** are all abstract. They each declare at least one pure virtual function (= 0), preventing direct instantiation.

```
class Person {
    virtual void displayInfo() const = 0; // pure virtual };
class LibraryItem {    virtual void checkout() = 0; };
```

10. Runtime Polymorphism

In main.cpp (Module 1 demo), an array of **Person*** pointers holds **Student**, **GradStudent**, **Faculty**, and **Staff** objects. Calling `people[i]->displayInfo()` invokes the correct overridden method via the virtual dispatch table.

```
Person* people[4] = { students[0], grads[0],
faculty[0], staff[0] }; for (int i=0;i<4;i++)
    people[i]->displayInfo(); // virtual dispatch
```

11. Operator Overloading

Course overloads == (compare by course code), << (stream insertion via friend), and + (merge two waiting lists). **FeeRecord** overloads -= to record a fee payment and update the balance.

```
bool Course::operator==(const Course& o) const {
    return courseCode == o.courseCode; } FeeRecord&
FeeRecord::operator-=(double payment) {
    totalPaid += payment; balance -= payment;
    return *this; }
```

12. Friend Functions

The operator<< for **Course** and **Invoice** is declared friend so it can access private members (courseCode, items array) while remaining a non-member function compatible with ostream.

```
class Course {
    friend ostream& operator<<(ostream& os, const Course& c);
};
ostream& operator<<(ostream& os, const Course& c) {
    os << c.courseCode << " - " << c.courseName;
    return os;
}
```

13. Static Members

Invoice has a static int invoiceCounter that is shared across all Invoice objects. Each constructor call increments it via invoiceID = ++invoiceCounter, ensuring unique sequential IDs without external state.

```
class Invoice {
    static int invoiceCounter; // shared
public:
    static int getInvoiceCount() { return invoiceCounter; }
};
int Invoice::invoiceCounter = 0; // defined in .cpp
```

14. Copy Constructor (Deep Copy)

FeeRecord dynamically allocates a string* remarks array. The copy constructor manually allocates a fresh array and copies each element, so modifying one object's remarks does not affect the other.

```
FeeRecord::FeeRecord(const FeeRecord& other) {
    remarkCapacity = other.remarkCapacity;
    remarks = new string[remarkCapacity];
    for (int i=0;i < remarkCapacity;i++) remarks[i] = other.remarks[i];
}
```

15. Copy Assignment & Destructor (Rule of Three)

FeeRecord and **Invoice** both define a copy assignment operator (with self-assignment guard and deep copy) and a destructor (delete[]). This satisfies the C++ Rule of Three for classes that manage heap resources.

```
FeeRecord& FeeRecord::operator=(const FeeRecord& o)
{
    if (this == &o;) return *this; // guard
    delete[] remarks;
    remarks = new string[o.remarkCapacity];
    ...
    return *this;
}
```

16. Search Functions

Library::searchByTitle() performs a linear search over the catalog array.

Reports::findStudentByRollNo() searches a Student* array by roll number.

```
LibraryItem* Library::searchByTitle(const string& t) const {
    for (int i=0;i          if (catalog[i]->getTitle() == t)
    return catalog[i];      return nullptr; }
}
```

17. Array-based Collections

Library stores its catalog as LibraryItem* catalog[100] and issued records as a plain IssuedRecord struct array. No STL containers are used, per the assignment requirements.

```
class Library {      LibraryItem*
catalog[MAX_CATALOG_SIZE];      IssuedRecord
issuedRecords[MAX_ISSUED_ITEMS];      int
catalogSize, issuedCount; };
```

18. Arrays of Objects

Global arrays Student* students[20], Faculty* faculty[10], and Course* courses[10] in main.cpp hold all campus entities. Room occupant arrays, HostelBlock room arrays, and Student enrolled course arrays are further examples throughout all modules.

```
Student* students[MAX_STUDENTS];
Faculty* faculty[MAX_FACULTY];
Course*  courses[MAX_COURSES];
```

19. Exception Handling

Four custom exception classes are defined in Exceptions.h: **CapacityExceededException**, **OverdueException**, **InvalidPaymentException**, and **RoomFullException**. All inherit from std::exception and are thrown with throw and caught with try/catch.

```
try {
    courses[0]->enrollStudent(s); // throws if full }
catch (const CapacityExceededException& e) {      cout <<
e.what() << endl;      courses[0]->addToWaitingList(s); }
```

20. File I/O (fstream)

Library::saveCatalog() serialises the catalog to data/library_catalog.txt as pipe-delimited lines using ofstream. **Library::loadCatalog()** reads it back with ifstream and reconstructs **Book** and **Journal** objects dynamically.

```
void Library::saveCatalog() const {
    ofstream f(catalogFile);
    for (int i=0;i          f << catalog[i]->getType() << "|"
    << catalog[i]->getTitle() << "\n";
    }
}
```

21. Reporting & Utilities

Reports.h / Reports.cpp (Module 6) provides static helper methods: printStudentList(), printFacultyList(), printCourseList(), and generateCampusReport() which writes the full campus state to a text file.

```
Reports::generateCampusReport(
students,          studentCount,
faculty,           facultyCount,
courses, courseCount, lib,
hostelMgr );
```

22. Memory Management

All dynamically allocated objects (Student, Faculty, Course, Room, HostelBlock, LibraryItem subclasses) are freed via delete either in destructors or in the cleanupGlobals() function at the end of main().

```
void cleanupGlobals() {
    for (int i=0;i          delete students[i];      for
(int i=0;i          delete courses[i]; }
```

23. Sorting and Searching

Reports::sortStudentsByGPA() implements selection sort (descending).

Reports::sortStudentsByName() implements bubble sort (ascending).

Reports::findStudentByRollNo() performs linear search.

```
void Reports::sortStudentsByGPA(Student* arr[], int n) {
    for (int i=0;i          int max=i;
        for (int j=i+1;j          if (arr[j]->getGPA()>arr[max]-
>getGPA())          max=j;          swap(arr[i],arr[max]);      }
}
```

24. Composition

HostelBlock owns an array of Room* objects and deletes them in its destructor — classic composition (whole controls the lifetime of its parts). **HostelManager** similarly owns its HostelBlock* array.


```

class HostelBlock {      Room*
rooms[MAX_ROOMS_PER_BLOCK]; public:
    ~HostelBlock() {

        for (int i=0;i      } };;

```

25. Aggregation

Course holds a **Faculty*** reference to its instructor but does NOT delete the **Faculty** object — **Faculty** lives independently. This is classic aggregation: **Course** uses **Faculty**, but does not own it.

```

class Course {      Faculty* instructor; // aggregation
public:      Course(..., Faculty* instr) :
instructor(instr) {}      // No delete in destructor for
Faculty*
};

```

4. Module Descriptions

Module 1 — Person Hierarchy

The backbone of the system. **Person** is an abstract base class with four attributes (name, CNIC, age, contact) and a pure virtual `displayInfo()`. **Student**, **Faculty**, and **Staff** each inherit directly (single inheritance). **GradStudent** extends **Student** (multi-level). At runtime, a **Person*** array dispatches `displayInfo()` via virtual dispatch.

- **Person** (abstract) → **Student** → **GradStudent** (multi-level)
- **Person** (abstract) → **Faculty**
- **Person** (abstract) → **Staff**
- `displayInfo()` called via **Person*** array (runtime polymorphism)

Module 2 — Course & Enrollment

Course tracks enrollment with a **Student*[50]** array and a waiting list. `enrollStudent()` throws **CapacityExceededException** when full. Three operators are overloaded: `==` for equality by course code, `<<` via friend for printing, and `+` for merging waiting lists. **Enrollment** records a student-course pair with date and grade.

- Operator overloading: `==`, `<<`, `+`
- Friend function: `operator<<`
- **CapacityExceededException** thrown and caught
- **Enrollment** aggregates **Student*** and **Course***

Module 3 — Library System

LibraryItem is an abstract base with `checkout()` and `returnItem()` pure virtual. **Book** and **Journal** implement them. **Library** stores up to 100 items in a `LibraryItem*` array. `saveCatalog()` writes to `data/library_catalog.txt` via `ofstream`; `loadCatalog()` reads it back. `Overdue` returns throw **OverdueException** with a computed fine amount. The destructor frees all heap-allocated items.

- Abstract class hierarchy: `LibraryItem` → `Book` / `Journal`
- `fstream` read/write persistence
- `OverdueException` with fine calculation
- Destructor manages heap-allocated catalog items

Module 4 — Fee & Finance

FeeRecord models a student's complete fee breakdown. Its copy constructor and copy assignment perform a deep copy of the dynamically allocated `string*` `remarks` array. `operator=` records a payment and updates the balance, throwing **InvalidPaymentException** on invalid amounts. **Invoice** uses a static `invoiceCounter` for auto-generated IDs and a dynamic `InvoiceItem*` array freed in its destructor.

- `FeeRecord`: copy constructor, copy assignment (Rule of Three)
- `operator=` for payment recording
- `Invoice`: static `invoiceCounter`, destructor
- `InvalidPaymentException` thrown on bad amounts

Module 5 — Hostel Management

Manageable is a virtual base class. **Accommodation** and **Reportable** both inherit from it with virtual — this creates the diamond problem, resolved automatically. **HostelManager** inherits from both (multiple inheritance) and provides concrete implementations of `allocateRoom()` and `generateOccupancyReport()`. **HostelBlock** composes **Room** objects; **HostelManager** composes **HostelBlock** objects. Both destructors free owned memory.

- Multiple inheritance: `HostelManager` : `Accommodation`, `Reportable`
- Virtual inheritance (diamond problem) via `Manageable`
- Composition: `HostelManager` owns `HostelBlock`; `HostelBlock` owns `Room`
- `RoomFullException` thrown when room capacity is exceeded

Module 6 — Reports & Utilities

The **Reports** class provides static utility methods for the entire system. Sorting is done manually (selection sort by GPA, bubble sort by name). Linear search finds students by roll number. `generateCampusReport()` prints a formatted summary to the console and writes it to `data/campus_report.txt` via `ofstream`. `Exceptions.h` centralises all four custom exception classes.

- `sortStudentsByGPA()` — selection sort descending
- `sortStudentsByName()` — bubble sort ascending

- findStudentByRollNo() — linear search
- generateCampusReport() — console + file output

5. GitHub Workflow

The repository is hosted at <https://github.com/abdullahbutt78/HITEC-OOP-SCMS-25-CS-078>. The project was committed in two phases as required:

- **Phase 1 commit:** "Add Phase 1 files - Person, Course, and Enrollment classes"
- **Phase 2 commit:** "Add Phase 2 files - Library, Finance, Hostel, and Reporting modules"
- **Final commit:** "Final submission - all modules complete with UML and report"

A GitHub Actions workflow (.github/workflows/build.yml) automatically compiles the project on every push using `g++ -std=c++17 -Wall`. This provides continuous integration and ensures the project always builds cleanly from a fresh checkout.

```
name: Build SCMS on:
[push, pull_request]
jobs:  build:
    runs-on: ubuntu-latest
steps:  - uses:
actions/checkout@v3      -
name: Compile
    run: g++ -std=c++17 -Wall src/**/*.cpp src/main.cpp -o scms
```

6. Challenges & Solutions

Challenge 1: Diamond Problem with Multiple Inheritance

Problem: When HostelManager needed to inherit from both Accommodation and Reportable, and both of those inherited from Manageable, the compiler would produce two copies of Manageable's subobject inside HostelManager — causing ambiguous member access errors.

Solution: Applied C++ virtual inheritance. By declaring class Accommodation : public virtual Manageable and similarly for Reportable, the compiler guarantees exactly one shared Manageable subobject in any derived class. HostelManager overrides getManagerName() once, resolving all ambiguity.

Challenge 2: Deep Copy for FeeRecord's Dynamic remarks Array

Problem: FeeRecord initially used a fixed-size string array. When requirements grew to support a variable number of remarks, the array was changed to string* remarks allocated on the heap. Shallow copy (the compiler default) would cause two FeeRecord objects to share the same remarks pointer, leading to double-free on destruction.

Solution: Implemented the full Rule of Three: a custom copy constructor that new[]-allocates a fresh array and copies elements, a copy assignment operator with a self-assignment guard that

frees the old array before deep-copying, and a destructor that calls `delete[]` remarks. This ensures complete independence between copied objects.

Challenge 3: NAME_CLASH between header constants

Problem: Both `Student.h` and `main.cpp` defined `const int MAX_COURSES = 10`. When both were included in the same translation unit, the compiler raised a redefinition error.

Solution: Renamed the constant in `Student.h` to `MAX_ENROLLED_COURSES` to accurately describe its domain (maximum courses a single student may enroll in) and avoid collision with the system-wide `MAX_COURSES` in `main`. This also reinforced the principle of using descriptive, unambiguous names.

7. Conclusion

The Smart Campus Management System successfully demonstrates all 25 mandatory OOP concepts from CS-104L in a single, cohesive C++17 application. Designing the Person hierarchy first established the polymorphic foundation that all other modules build on. Implementing the diamond problem with virtual inheritance in the hostel module was particularly instructive — it made concrete a concept that is easy to read about but requires hands-on experience to truly understand.

Working through the Rule of Three for `FeeRecord` reinforced why C++ forces explicit thinking about resource ownership: the compiler's default shallow-copy behaviour is correct only when a class does not manage heap resources. The dynamic remarks array made this immediately obvious when an early test caused a double-free crash.

Version-controlling the project on GitHub and writing a GitHub Actions CI workflow added professional discipline: every push triggers a clean compile, making regressions impossible to miss. This project has been an end-to-end simulation of real software development workflow.